

# Sustainability implications of Large Language Model use in Model-Driven Engineering

1<sup>st</sup> Kevin Lano  
Informatics Dept.  
King's College London  
London, UK  
kevin.lano@kcl.ac.uk

2<sup>nd</sup> Shekoufeh Kolahdouz Rahimi  
School of Computing  
University of Roehampton  
London, UK  
shekoufeh.rahimi@roehampton.ac.uk

3<sup>rd</sup> Qiaomu Xue  
Informatics Dept.  
King's College London  
London, UK  
qiaomu.l.xue@kcl.ac.uk

**Abstract**—Large language models (LLMs) have been increasingly used for a wide range of software engineering processes, including model-driven engineering (MDE). LLMs have been used to perform reverse engineering of code to models, to generate code from models, and to perform model transformations, as alternatives to rule-based techniques for these activities. In this paper we examine the energy-use implications of applying LLMs for code generation and reverse engineering within an MDE process. We also consider LLM capabilities for optimising software energy use.

**Index Terms**—MDE, LLM, code generation, reverse engineering, sustainable software.

## I. INTRODUCTION

The advent of Large Language Models (LLMs) as a general purpose deep learning (DL) technique has transformed software development practice, with tools such as Copilot and Cursor being widely-used to produce program code based on natural language requirements and partial specifications, such as the intended results of test cases. This approach is highly usable for practitioners; however, the reliability and accuracy of LLM-based code generation are still open to question [9], [15], [29], [32], [42]. LLMs have also been used for reverse engineering of programs to specifications [4], [6], [34]–[36], [41] and for program translation [1], [13], [21], [24], [26]. In each of these application domains, the high usability of the conversational interfaces offered by LLM-based tools often outweighs (at least for non-critical applications) the stochastic and black-box nature of LLMs. An issue that has been less widely considered is the energy use of LLM-based tools, or the operational energy use of software produced by LLM-based code generators. In this paper we investigate this aspect, and compare the energy use of LLM-based software engineering techniques with that of MDE techniques.

### A. Research questions

We consider the following specific issues:

- 1) How do MDE code generation approaches compare to manual and LLM-based code production approaches, with regards to the energy use of the code production process?
- 2) How do MDE reverse engineering approaches compare to LLM-based reverse engineering approaches, with regards to the energy use of the code abstraction process?

- 3) How effective are LLMs at detecting and correcting energy-efficiency flaws in code, compared to MDE techniques for energy-use analysis and improvement?

## II. CODE PRODUCTION APPROACHES

Three code production practices are considered here: (i) manual code production, using IDEs such as Visual Studio; (ii) automated code generation from software models, using a model transformation (MT) or (iii) synthesis of code by an LLM, based on user prompts and contextual information.

### A. Manual code production

Traditional program development is a manual process, using development environments (IDEs) such as Visual Studio (VS), Xcode or Android Studio. This gives the developer the most control over the form of the code, however the human resources required can be considerable. From an energy-use perspective, the IDEs are usually large, complex and energy-intensive pieces of software. Using the Green Algorithms calculator<sup>1</sup> we estimated that VS<sup>2</sup> uses 321mWh/minute for initialisation on the Windows 10 platform, and 319mWh/minute for typical editing/compiling/debugging operations [19]. The observed values for Android Studio and Xcode (on MacOS) were even higher.

### B. Automated code generation using MDE

One of the key motivations of MDE was to automate code production, thus reducing development time and resources, and improving the rigour of development. This is still perceived as an important benefit of MDE by practitioners [2]. Model transformation languages specialised for code generation include EGL [11] and CSTL [23]. These are model-to-text (M2T) transformation languages, generating executable code text from model content according to precise rules. The source models can either be models of a general-purpose modelling language such as UML/OCL, or models in a domain-specific language (DSL).

<sup>1</sup>calculator.green-algorithms.org

<sup>2</sup>VS 2019, without embedded LLM coding support.

### C. Code production using LLMs

In contrast to the MDE code production approach, LLMs do not use fixed rules for code generation, but instead apply their embedded language knowledge. This can enable them to produce output for inputs which a fixed rule-based system cannot, because the rules do not cover the specific case. On the other hand, the output is less predictable and typically needs human validation and post-processing before it can be used for production code.

The survey [15] examines LLM performance in code generation, they conclude that there has been substantial recent improvements in LLM capabilities, but emphasise the importance of training dataset quality for an LLM's ability for accurate code generation. The study [9] identifies that LLMs performance for code generation decreases with the scale of the input, so that LLMs perform better at generating individual statements and methods, rather than complete semantically-coherent classes.

## III. REVERSE ENGINEERING APPROACHES

Model-driven reverse engineering (MDRE) involves the use of code-to-model transformations, often based on traversing abstract syntax trees or parse trees of source code, to produce abstracted models of the software in a suitable modelling language [8], [37]. Mainstream industry practice for reverse engineering is also based on customised rulesets for code abstraction, but these are typically coded in a programming language such as Java, rather than a MT language [27], [39]. Fixed rulesets are a more significant limitation in the case of reverse engineering, compared to the code-generation case, because of the high variety of source code that could be input to reverse engineering. The use of LLMs for reverse engineering could help to address this limitation, but LLM application for reverse-engineering models from code has only recently been explored [4], [6], [34]–[36]. In [6], GPT-4 is used for reverse engineering UML models from code, and this LLM is shown to have the capability for imitating human code abstraction techniques. The baseline LLM capabilities for UML and OCL abstraction from code are evaluated in [35], [36]. They find that the surveyed LLMs have limited OCL knowledge in particular, which leads to syntactic and semantic errors in abstracted models. However, [34] shows that fine-tuning of an LLM based on a dataset produced by a precise MDRE approach can substantially improve the LLM reverse-engineering accuracy.

## IV. ENERGY-USE FLAW ANALYSIS AND CORRECTION

Although LLMs have been used extensively for software development tasks [12], [43], there is a gap in the application of LLMs for the analysis and improvement of software sustainability. LLMs have been used for efficiency improvement of business systems [38], but have not been applied for sustainability improvement. MDE techniques for energy-use analysis and reduction were introduced in [18], [20], [22]. Identified energy-use flaws include:

- Duplicated expression evaluations – cases where there are cloned expressions within an operation, which have the same value, or where there is a loop constant expression that could be evaluated repeatedly within an iteration/loop, and whose value is independent of the iterator/loop variables.
- Inefficient use of data structures – for example, using a sequence to implement a unique-element collection.
- Redundant operation results – applying an operation that returns a tuple of results, but only using one of these.
- Inefficient algorithm choices – such as using bubble sort, using recursion instead of iteration, or using a loop construct where a non-iterative computation is available instead.

Detection rules for these flaws are built-in to the AgileUML MDE toolset [10], together with refactoring transformations to remove the flaws, however there is substantial variation in how such flaws can manifest in code, so that fixed rules may be insufficient to detect all cases. In addition, many of the refactorings need to be manually selected. Thus an LLM-based energy-flaw analysis and improvement approach could potentially provide more generalised flaw detection and more automated refactoring support.

## V. EVALUATION

In this section we perform quantitative experiments to measure the energy use of different code generation and code abstraction techniques, and to evaluate the effectiveness of MDE and LLM approaches to produce energy-efficient code.

We evaluate software energy use in milli-Watt hours (mWh) using the calculator at <https://calculator.green-algorithms.org>, based upon CPU utilisation, execution time and memory use. The implementation platform is Windows 10 OS on a laptop with 4 core i5-9400 processor, UK location, 8GB available memory. We consider three implementation programming languages: Java, JavaScript and Python. AgileUML is run in graphical UI mode using Java version 1.8.0\_261 on the Windows 10 platform. For each data point, the average of 3 energy-use measurements is taken. Evaluation data is available at Zenodo.org<sup>3</sup>.

### A. Evaluation of code generation

To test the relative energy use of different code production approaches (manual, LLM and MDE) we apply these approaches to four examples of UML/OCL specifications of varying sizes, where size is measured as the number of words/symbols in the source. The examples are taken from [19], and we also take the estimate of manual development energy use from this paper. Code for the cases is generated in JavaScript, using the MDE and LLM code generators of [42] (Table I). The MDE energy use is computed using the Green Algorithms calculator for code generator execution on

<sup>3</sup><https://zenodo.org/records/16728657>

the Windows 10 platform, the LLM energy use for the Llama2-7B LLM is estimated based on the figure of 1.6mWh per input token given by [3].

TABLE I  
CODE GENERATION ENERGY USE (MWh)

| Case Study | Size (tokens) | Approach    |        |       |
|------------|---------------|-------------|--------|-------|
|            |               | Manual (VS) | LLM    | MDE   |
| mmUML      | 114           | 9891        | 182.4  | 0.016 |
| OclDate    | 1183          | 19461       | 1892.8 | 2.57  |
| MatrixLib  | 1824          | 29031       | 2918.4 | 11.7  |
| Excel      | 4227          | 38601       | 6763.2 | 17.42 |
| Averages   | 1837          | 24246       | 2939.2 | 7.93  |
| Std. Dev.  |               | 10699       | 2414   | 6.99  |

The results show a consistent difference between the three code production approaches, with manual code production 8.2 times as energy-expensive than LLM production on average, and LLM production in turn over 350 times as energy-expensive as MDE code generation on average. Although these results seem to contradict the conclusions of [7] that LLM code synthesis is more expensive than manual coding, here we use a ‘single shot’ LLM code production approach, whilst [7] use an iterative approach, with the LLM applied repeatedly until a satisfactory result is produced. The IDE execution energy use of manual coding is also not considered in [7]. If the figure of 104mWh per LLM inference is used from [25], then the ordering of the three approaches in terms of energy use does not change, although the LLM usages are reduced. Likewise, if we assume 5 applications of the LLM per code generation, the relative ordering of energy usages does not change.

#### B. Evaluation of code abstraction

To test the relative energy use of different code abstraction approaches (using a model transformation, Java coded AST processing or an LLM) we apply these approaches to four examples of Java and Python programs specifications of varying sizes (Tables II, III).

TABLE II  
JAVA ABSTRACTION ENERGY USE (MWh)

| Case Study         | Size (tokens) | Approach |      |        |
|--------------------|---------------|----------|------|--------|
|                    |               | MT       | Java | LLM    |
| Avatar case 113    | 73            | 13.9     | 1.56 | 116.8  |
| nsapp              | 425           | 11.4     | 0.89 | 680    |
| Transcoder case 20 | 909           | 25.9     | 4.63 | 1454.4 |
| CDO app            | 748           | 16.6     | 2.8  | 1196.8 |
| Averages           | 539           | 16.95    | 2.47 | 862    |
| Std. Dev.          |               | 5.5      | 1.4  | 513    |

The Java cases are taken from the widely-used Avatar [1] and Transcoder datasets [16], together with two extracts of a real-world finance application. The Python cases are also taken from a finance application, together with two synthetic examples. The energy use of the MT-coded and Java-coded abstraction transformations applied to the Java cases, and of the MT-coded abstractor applied to the Python cases are computed using the Green Algorithms calculator, the energy

use of the LLM-based abstraction for all cases is computed using the figure of 1.6mWh per input token given for Llama2-7B by [3].

TABLE III  
PYTHON ABSTRACTION ENERGY USE (MWh)

| Case Study           | Size (tokens) | Approach   |             |
|----------------------|---------------|------------|-------------|
|                      |               | MT         | LLM         |
| Bonds functions 1    | 209           | 18.4       | 334.4       |
| Bonds functions 2    | 239           | 18.4       | 382.4       |
| py300                | 563           | 30.2       | 900.8       |
| py500                | 971           | 50.3       | 1553.6      |
| Averages (Std. Dev.) | 495.5         | 29.33 (13) | 792.8 (492) |

Overall these results show a consistent pattern, where for Java abstraction the Java-coded transformation is about 7 times more energy-efficient than the MT-coded transformation on average, and this is about 50 times more energy-efficient than the LLM abstractor. In the Python cases the MT-coded abstractor is about 27 times more energy-efficient on average than the LLM abstractor.

#### C. Energy-use analysis and improvement

To investigate this aspect, we define six example programs which each exhibit one energy-use flaw as described in Section IV. We then apply an MDE approach (AgileUML) to analyse and refactor the examples, and the GPT-4 LLM. The examples are: (i) misuse of a sequence type as a set; (ii) repeated evaluation of a loop-constant expression; (iii) bubble sort; (iv) a spurious loop (the sum of the first N integers); (v) recursion (factorial); (vi) redundant operation results.

We evaluate whether the analysis/improvement approach can successfully identify the flaws, and if it can correct them. If a correction is performed, the percentage energy-use reduction achieved by the improved program is also evaluated, computed as  $100 * \frac{\text{original} - \text{refactored}}{\text{original}}$  where *original* and *refactored* are the sum of energy usages of the original and refactored codes executing a set of common test cases (Table IV).

TABLE IV  
JAVA ENERGY USE ANALYSIS/IMPROVEMENT USING MDE

| Flaw case                   | Detected? | Corrected? | Improvement |
|-----------------------------|-----------|------------|-------------|
| 1. Sequence as set          | ✓         | ×          | —           |
| 2. Loop-constant expression | ✓         | ✓          | 90%         |
| 3. Bubble sort              | ✓         | ×          | —           |
| 4. Unnecessary loop         | ✓         | ✓          | 100%        |
| 5. Recursion                | ✓         | ✓          | 92%         |
| 6. Redundant results        | ✓         | ×          | —           |

AgileUML detects the energy-use flaw in each case, and advises suitable correction actions. For cases 2, 4 and 5 there are built-in refactorings provided: *Extract local variable*; *Simplify OCL* and *Replace recursion by iteration*, respectively, but these need to be selected manually. The refactored code is substantially more energy-efficient in each of these cases. For cases 1, 3, 6, the flaw is detected, but there is no simple refactoring to remove the flaw.

We also apply GPT-4 to the same source programs and tasks, using the prompt:

Please analyse the following Java program, to identify flaws that could cause excess energy use by the program at runtime.

Generate an equivalent Java version of the program with the flaws removed.

The results are shown in Table V.

TABLE V  
JAVA ENERGY USE ANALYSIS/IMPROVEMENT USING GPT-4

| Flaw case                   | Detected? | Corrected? | Improvement |
|-----------------------------|-----------|------------|-------------|
| 1. Sequence as set          | ✓         | ✓          | 89%         |
| 2. Loop-constant expression | ✓         | ✓          | 88%         |
| 3. Bubble sort              | ✓         | ✓          | 99%         |
| 4. Unnecessary loop         | ✓         | ✓          | 100%        |
| 5. Recursion                | ✓         | ✓          | 71%         |
| 6. Redundant results        | ✓         | ×          | —           |

It can be seen that GPT is more effective at energy-use improvement than the MDE approach, at least for the Java language and on these relatively simple examples. It is already aware that operation calling and recursion should be minimised to improve energy-efficiency, and that redundant computations should be avoided. It detects the redundant result issue in case 6 but the proposed solution still has this problem.

#### D. Summary

With regard to the research questions of Section I-A, we found that:

- 1) MDE code generation approaches appear to be considerably more energy efficient than manual or LLM-based code production approaches, by at least 2 orders of magnitude for the examples considered.
- 2) MDE reverse engineering approaches, either MT-based or coded directly in Java, are also substantially more energy-efficient than LLM-based reverse engineering, for the examples considered, again with a factor of 2 orders of magnitude efficiency gain for the Java-coded abstractor compared to the LLM abstractor.
- 3) The GPT-4 LLM was able to outperform the MDE rule-based tools for detecting and correcting energy-efficiency flaws in Java code, and appeared to have good knowledge of the issues involved.

The relative order of MDE and LLM energy use is not changed if alternative LLM energy use estimates, such as those of [25], are adopted. One threat to validity is that LLM responses may change over time [33]. We carried out the experiments of Table V on the 27th–28th June 2025. Subsequently we repeated the experiments on 2nd August 2025 with similar results.

## VI. RELATED WORK

There have been many studies evaluating the accuracy and reliability of LLMs for code generation and program translation tasks, such as [1], [9], [13], [15], [26], [29]. The

papers [21], [42] specifically compare MDE and LLM-based techniques for program translation (in the COBOL to Java case) and code generation (for JavaScript and Kotlin code generation) respectively. However these works only consider accuracy and usability of the generation and translation processes, and do not compare their energy use. The work of [32] integrates LLM-based code generation into an MDE process, but also does not consider the energy-use implications of the approach. The study of [7] compares LLM-assisted code development using GPT-4 with manual coding. They find that the LLM approach consumes an average of 67.1Wh for 12 Python programming tasks, versus 2.035Wh for manual development of the same tasks. In contrast, we found that single-shot LLM code generation using a smaller (Llama2-7B) LLM can be more efficient than manual production (Section V-A). Our estimates for the energy-use of manual coding are higher (an average of 24.246Wh) than those given in [7], but [7] do not include the energy consumption of an IDE in their estimates.

The application of LLMs in MDE is considered by the survey of [30], but they do not consider the energy-use implications of this use. The improvement of the energy efficiency of MDE processes, particularly of model transformation execution, is considered by [19], [31]. They define techniques for energy use reduction in MDE processes, and evaluate the impact of these techniques in practice.

The precise evaluation of the energy use of LLM training or inference is itself a complex problem. The paper [3] performs a detailed energy-use analysis of LLM inference for several widely-used models, including the Llama2-7B LLM used by [42] for code generation, and surveyed by [35], [36] for reverse-engineering. The paper [3] estimates that an average of 5.61J is used per input token for inference by this model. This is 1.6mWh per token, the figure we used for LLM inference in Sections V-A, V-B. However, in contrast, the average energy use per inference on a similar 7B LLM is estimated in [25] as 104mWh. But they do not state what is considered as an average inference task, in terms of numbers of input tokens.

MDE-based techniques for improving software energy-efficiency are described in [18], [20], [22]. These adopt concepts of *software energy-use flaw* and *sustainability debt*, by analogy with *software quality flaw* and *technical debt*, and define energy-flaw indicators and refactorings to detect and remove such flaws. A similar approach is adopted for the mobile app domain in [5]. The use of MDE to define energy-efficient software architectures is considered by [17], [28]. However, none of these works investigate the application of LLMs for software sustainability analysis/improvement.

The quality of LLM-generated or translated code is evaluated by [13], [21], [32]. They find that the cyclomatic complexity and other quality measures of LLM-generated/translated code can be acceptable, but no evaluation of the energy-efficiency of the produced code is carried out. In [14], the authors evaluate the capabilities of LLMs for generating efficient code (in terms of code execution time). They find that the baseline ability of LLMs for efficient code production

are not good, but that their capabilities can be enhanced by suitable additional training. The work of [40] evaluates the energy-efficiency of code generated by Copilot and ChatGPT against that of human-coded solutions. They find that for the majority of 6 programming problems (including sorting) the human-coded solution is more energy-efficient: in 5 of 6 cases for Copilot and in 3 of 6 cases for ChatGPT. However they do not consider the use of an LLM to analyse and improve the energy efficiency of human-coded solutions.

## CONCLUSIONS AND FUTURE WORK

In this paper we have considered the energy-use implications of using LLMs as alternative tools for code production and code abstraction within an MDE process, and we also investigated LLM use for the improvement of software energy-efficiency. We found that overall an LLM is substantially more energy-expensive than a corresponding rule-based MDE approach, however there may be tradeoff benefits in LLM use, in terms of their improved usability and ability to handle a wide range of inputs, compared to the MDE approach. We also found that GPT-4 was able to effectively analyse and correct simple energy-use flaws in Java code.

Future work in this direction will investigate fine-tuning of an open-source LLM for the energy-use analysis and improvement tasks, using training datasets produced using GPT-4 and the AgileUML energy-use analysis and refactoring tools.

## REFERENCES

- [1] W. Ahmad, M. Tushar, S. Chakraborty, and K-W. Chang. AVATAR: a parallel corpus for Java-Python program translation. *arXiv:2108.11590v2*, 2023.
- [2] H. Alfraihi and K. Lano. Trends and insights into the use of model-driven engineering: a survey. In *SAM/MODELS*, 2023.
- [3] M. Argerich and M. Patino-Martinez. Measuring and improving the energy efficiency of large language models inference. *IEEE Access*, 12, 2024.
- [4] A. Boronat and J. Mustafa. MDRE-LLM: A tool for analysing and applying LLMs in software reverse engineering. In *SANER '25*. IEEE, 2025.
- [5] L. Brunschwig and O. Le Goar. Cross-detection of mobile-specific energy hotspots: MBSE to the rescue. In *SusMOD Workshop, MODELS 2024*, 2024.
- [6] V. Campenello, S. Shahbaz, V. Indykov, and D. Struber. On the use of GPT-4 in the reverse engineering of class diagrams. *JoT*, 2025.
- [7] K. Cheung, M. Kaul, G. Jahangirova, M. Mousavi, and E. Zie. Comparative analysis of carbon footprint in manual vs LLM-assisted code development. *ArXiv:2505.0521v1*, 2025.
- [8] G. Deltombe, O. Le Goar, and F. Barbier. Bridging KDM and ASTM for model-driven software modernization. In *SEKE 2012*, 2012.
- [9] X. Du, M. Liu, K. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou. Evaluating large language models in class-level code generation. In *ICSE 2024*, 2024.
- [10] Eclipse. 2025. Agile UML project, <https://projects.eclipse.org/projects/modeling.agileuml>.
- [11] Epsilon project, 2024. Epsilon Generation Language, <https://projects.eclipse.org/projects/modeling.epsilon>. Accessed 26.3.2024.
- [12] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. Zhang. Large language models for software engineering: Survey and open problems. In *ICSE '23*, 2023.
- [13] S. Gandhi, M. Patwardhan, J. Khatri, L. Vig, and R. Medicherla. Translation of low-resource COBOL to logically-correct and readable Java leveraging high-resource Java refinement. In *LLM4Code*, 2024.
- [14] D. Huang, J. Dai, H. Weng, P. Wu, Y. Qing, H. Cui, Z. Guo, and J. Zhang. EFFI-LEARNER: Enhancing efficiency of generated code via self-optimisation. In *38th NeurIPS*, 2024.
- [15] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim. A survey on large language models for code generation. *arXiv*, 2024.
- [16] M-A. Lachaux, B. Roziere, L. Chanussot, and G. Lample. Unsupervised translation of programming languages. *arXiv:2006.03511v3*, 2020.
- [17] K. Lano, L. Alwakeel, and Z. Rahman. Design patterns for software sustainability. In *Pattern Languages of Programs (PLoP)*, 2024.
- [18] K. Lano, L. Alwakeel, and Z. Rahman. Software modelling for sustainable software engineering. In *2nd Agile MDE Workshop, STAF 2024*, 2024.
- [19] K. Lano and S. Rahimi. Sustainability analysis and improvement of model driven engineering and model transformation languages. In *IKT 2024*, 2024.
- [20] K. Lano, S. Rahimi, and Z. Rahman. Specification and design refactorings for sustainable agile model-driven engineering. In *AgileMDE 2025, STAF 2025*, 2025.
- [21] K. Lano, H. Siala, and K. Jin. Comparing LLM-based and model-driven program translation. In *ICoSSE*, 2025.
- [22] Kevin Lano, Shekoufeh Kollahdouz Rahimi, and Zishan Rahman. Using MDE to support sustainable software re-engineering. In *ECMFA*, 2025.
- [23] Kevin Lano and Qiaomu Xue. Agile specification of code generators for model-driven engineering. In *2020 15th International Conference on Software Engineering Advances (ICSEA)*, pages 9–15, 2020.
- [24] X. Li, S. Yuan, X. Gu, Y. Chen, and B. Shen. Few-shot code translation via task-adapted prompt learning. *Journal of Systems and Software*, 2024.
- [25] A. Luccioni et al. Power hungry processing: Watts driving the cost of AI deployment? In *FACCT '24*. ACM, 2024.
- [26] A. Malyaya, K. Zhou, B. Ray, and S. Chakraborty. On ML-based program translation: perils and promises. *arXiv:2302.10812v1*, 2023.
- [27] A. De Marco, V. Iancu, and I. Asinofsky. COBOL to Java and newspapers still get delivered. In *Proceedings IEEE International Conference on Software Maintenance and Evolution*, pages 583–586. IEEE Press, 2018.
- [28] G. Morais, M. Adda, and D. Bork. Breaking down barriers: Building sustainable microservices architectures with model-driven engineering. In *SusMOD Workshop, MODELS 2024*, 2024.
- [29] F. Rabbi, Z. Ding, and J. Yang. A multi-language perspective on the robustness of LLM code generation. *arXiv:2504.19108v1*, 2025.
- [30] J. De Rocco, D. Di Ruscio, C. Di Sipio, P. Nguyen, and R. Rubel. On the use of large language models in model-driven engineering. *SoSyM*, 2025.
- [31] R. Rubel, J. di Rocco, and D. di Ruscio. On the energy-consumption of ATL transformations. *Software: Practice and Experience*, 2025.
- [32] A. Sadiq, S. Brulin, M. Olhofer, A. Ceravola, and F. Joublin. LLM as a code generator in agile model-driven development. *arXiv*, 2024.
- [33] J. Sallou, T. Durieux, and A. Panichella. Breaking the silence: the threats of using LLMs in software engineering. In *IEEE/ACM ICSE-NIER*, 2024.
- [34] H. Siala and K. Lano. Towards using LLMs in the reverse engineering of software systems to OCL. In *SANER 2025*, 2025.
- [35] H. Siala and K. Lano. Using LLMs to extract OCL specifications from Java and Python programs: an empirical study. In *OCL '25, STAF*, 2025.
- [36] H. Siala and K. Lano. Using LLMs to extract UML class diagrams from Java and Python programs: an empirical study. In *AMDE '25, STAF*, 2025.
- [37] H. Siala, K. Lano, and H. Alfraihi. Model-driven approaches for reverse engineering – a systematic literature review. *IEEE Access*, 2024.
- [38] V. Singh, C. Korlu, and W. Assuncao. Experiences on using LLMs to re-engineer a legacy system at Volvo Group. In *SANER '25*. IEEE, 2025.
- [39] H. Sneed. Migrating from COBOL to Java: A report from the field. In *IEEE Proc. of 26th ICSM*, pages 1–7. IEEE Press, 2011.
- [40] T. Vartziotis et al. Learn to code sustainability: An empirical study on LLM-based code generation. *ArXiv:2403.03344v1*, 2024.
- [41] D. Xie, B. Yoo, N. Jiang, M. Kim, L. Tan, X. Zhang, and J. Lee. How effective are LLMs in generating software specifications? In *SANER 2025*, 2025.
- [42] Q. Xue and K. Lano. Comparing LLM-based and MDE-based code generation for agile MDE. In *AgileMDE 2025, STAF 2025*, 2025.
- [43] Z. Zheng, K. Ning, Y. Wang, J. Zhang, D. Zheng, M. Ye, and J. Chen. A survey of LLMs for code. *arXiv*, 2024.